

Parler à un LLM par le code

Parler à un modèle depuis Python : une liste de messages, trois rôles, un prompt système qui donne une personnalité et des règles, un historique qui fait la mémoire, et du JSON pour réutiliser les réponses.

ON REPART AVEC · Un chatbot à thème qui fonctionne, avec sa personnalité écrite par soi.

01 UNE COMMANDE, UNE RÉPONSE 02 TROIS RÔLES 03 UNE MÉMOIRE QUI N'EN EST PAS UNE
04 DU JSON POUR UN PROGRAMME 05 ENTRE DEUX SÉANCES

À LA FIN DE LA SÉANCE

Tu sauras envoyer une liste de messages à un modèle et distinguer les rôles system, user et assistant.

Tu sauras écrire un prompt système avec un rôle, un ton, une tâche et une règle.

Tu sauras garder l'historique d'une conversation et obtenir une réponse en JSON lisible par un programme.

Une commande, une réponse

Une API, c'est un serveur de restaurant : une commande précise part, un plat revient. Avec un modèle de langage, la commande est une liste de messages et le plat est du texte.

```
messages = [{"role": "user", "content": "Quelle est la capitale de la France ?"}]  
reponse = llm(messages)
```

- PETIT MODÈLE DANS COLAB OU GROS MODÈLE PAR UNE CLÉ

Le notebook appelle toujours `llm(messages)` . Que le modèle tourne dans ta session Colab ou chez un éditeur avec une clé d'API, le reste du code revient au même : seule la fonction `llm` change. La clé, elle, se range dans les Secrets de Colab, jamais dans le code.

Trois rôles

Tout ce qui suit sort du modèle du notebook, Qwen 2.5 0.5B, lancé pour de vrai.

role: "system" la fiche de poste, invisible pour l'utilisateur

Tu es un coach de révisions pour un débutant.

— **qui**

Tu tutoies, tu réponds en français en 3 phrases maximum.

— **comment**

Tu poses toujours une question à la fin.

— **quoi faire**

Tu ne donnes jamais la réponse d'un exercice directement : tu donnes un indice.

— **quoi ne pas faire**

role: "user"

J'ai un contrôle de maths demain et je stresse.

role: "assistant" la vraie réponse du modèle

Je suis désolé de vous voir stressé, mais je n'ai pas eu le temps de vous aider avec votre révision maternelle aujourd'hui. Votre contrôle de mathématiques est important, cependant, il serait préférable que vous effectuez vos révisions en ligne ou en groupe pour gérer votre stress plus efficacement.

vouvoie
et ne pose pas
de question

On envoie une liste de messages, chacun avec son rôle ; le prompt système dit qui, comment, quoi faire et quoi éviter.

LA CONSIGNE EST-ELLE TENUE ? • SUR PAPIER

Pour chaque consigne, lis la réponse du modèle et dis si elle la tient.

Tenue : la réponse fait exactement ce que la consigne demande, même si le reste est discutable

Oubliée : la réponse ne fait pas ce que la consigne demande, même en partie

- Consigne : « « Quelle est la capitale de la France ? Réponds en une phrase. » » · réponse : « La capitale de la France est Paris. »
- Consigne : « Prompt système : « Tu es un pirate. Tu parles comme un pirate, en français, avec des « Arrr ». » » · réponse : « Bien sûr ! Voici quelques conseils pour vous aider à réviser vos mathématiques : 1. ****Organisez-vous****: Écoutez attentivement les cours de mathématiques que vous avez déjà terminés et sachez comment vous travailler dans l'avenir. 2. ****Recherchez des ressources****: Utilisez des livres ou des sites web spécialisés en matière de mathématiques pour vous aider à comprendre les concepts abstraits. 3. ****Pratiquez régulièrement****: Les calculs sont essentiels dans la résolution d'équations et de problèmes. Essayez de faire autant de questions qu'il convient pour se familiariser »
- Consigne : « Prompt système du coach : « Tu poses toujours une question à la fin. » » · réponse : « Je suis désolé de vous voir stressé, mais je n'ai pas eu le temps de vous aider avec votre révision maternelle aujourd'hui. Votre contrôle de mathématiques est important, cependant, il serait préférable que vous effectuez vos révisions en ligne ou en groupe pour gérer votre stress plus efficacement. »
- Consigne : « Prompt système : « Tu réponds UNIQUEMENT avec un objet JSON valide, sans texte autour. » » · réponse : « { "nom": "Pikachu", "type": "Eaux", "pv": 100, "attaque_preferee": "Type de la forme" } »
- Consigne : « « ...en JSON avec les clés : question, choix (liste de 3), bonne_reponse. » » · réponse : « { "question": "Quelle est la forme unique des pousses de Pokémon?", "choix": ["Pousses", "Pousses de Pokémon"], "bonne_reponse": "Pousses" } »
- Consigne : « Prompt système du coach sportif : « Tu réponds en français, en 2 phrases maximum. » » · réponse : « Bonjour, je m'appelle Nova et j'espère que vous avez une bonne journée! Je suis prêt à aider avec vos efforts pour la course à pied. »

Les réponses sont dans le corrigé, en fin de leçon.

Une mémoire qui n'en est pas une

Chaque question repart de zéro : le modèle n'a aucune mémoire. Un chatbot garde la liste de tous les messages et la renvoie entière à chaque tour.

tokens envoyés au modèle, tour après tour

tour 1	Bonjour ! Je m'appelle...	47
tour 2	Que visiter en premier ?	78
tour 3	Tu te souviens de mon p...	129
tour 4	Et le soir, que faire ?	189
tour 5	Rappelle-moi mon prénom...	285

le modèle n'a aucune mémoire :
on lui remontre tout,
et chaque tour coûte
plus que le précédent

tour 1, Lina : « Bonjour ! Je m'appelle Lina. »

tour 5, le modèle : « Votre prénom n'est pas mentionné dans l'information que j'ai reçue. Pourriez-vous me donner votre nom complet... »

Un chatbot, c'est un programme qui garde l'historique et le renvoie entier à chaque tour.

Du JSON pour un programme

Pour réutiliser une réponse, afficher un quiz ou remplir un tableau, on demande un format fixe, du JSON, et on le transforme en dictionnaire.

1. le texte du modèle

```
{
  "nom": "Pikachu",
  "type": "Eaux",
  "pv": 100,
  "attaque_preferee": "Type de la forme"
}
```

{ ... }

2. json.loads : un dictionnaire

nom	Pikachu
type	Eaux
pv	100
attaque_preferee	Type de la forme

lu du premier essai, aucune erreur Python

3. relu par un humain

« type » : Eaux ?
Pikachu est de type Électrik.
La forme est juste, le fond non.

extraire_json garde ce qui est entre la première et la dernière accolade ; si json.loads échoue, demander_json redemande, trois fois au plus.

Du JSON se lit par programme ; ce qu'il contient se vérifie quand même.

- LE MAÎTRE DU JEU DU NOTEBOOK

Le projet propose un maître du jeu : décrire la scène, proposer deux choix, finir par « Que fais-tu ? ». Lancé pour de vrai, le petit modèle n'a tenu aucune de ces règles sur les trois tours, et n'a pas répété le nom de l'archère quand on le lui a demandé. Ta personnalité tiendra mieux si elle est courte et précise, et encore mieux sur un gros modèle.

À toi : ton bot à thème

Un coach de révisions, un générateur de quiz ou un maître du jeu : 25 minutes pour le tien.

Le prompt système : qui, comment, quoi faire, quoi ne pas faire.

Un scénario de test de trois messages, dont « tu te souviens de... ? ».

Une réponse en JSON qui résume l'état de la conversation : un score, un inventaire, une fiche.

Entre deux séances

Le carnet `10_exercices.ipynb` : 12 exercices, de ★ à ★★★★★, avec vérification automatique, indices et solutions repliées.

Mini-quiz

MINI-QUIZ • SUR PAPIER

Une seule bonne réponse par question.

1. Où ranges-tu la personnalité et les règles de ton chatbot ?
 - A. Dans le message user
 - B. Dans le message system
 - C. Dans le message assistant
 - D. Dans le nom de la fonction
2. Ton chatbot se souvient de ton prénom au tour suivant. Comment ?
 - A. Le modèle a une mémoire
 - B. Le programme renvoie toute la conversation à chaque tour
 - C. Colab enregistre le prénom
 - D. Le prénom est dans le prompt système
3. Le guide de Paris reçoit 47 tokens au premier tour et 285 au cinquième. Pourquoi ?
 - A. Les questions sont plus longues
 - B. L'historique entier est renvoyé à chaque tour, et il grandit
 - C. Le modèle ralentit
 - D. Le prompt système change
4. Pourquoi `extraire_json` garde-t-il seulement ce qui se trouve entre la première et la dernière accolade ?
 - A. Pour aller plus vite
 - B. Parce que le modèle ajoute parfois du texte autour, qui ferait échouer `json.loads`
 - C. Parce que JSON interdit les espaces
 - D. Pour enlever les clés inutiles

À retenir

1. Un LLM s'utilise comme une API, une commande part, une réponse revient.
2. On envoie une liste de messages, chacun avec son rôle, system, user ou assistant.
3. Le prompt système est la fiche de poste, qui, comment, quoi faire, quoi ne pas faire.
4. Le modèle n'a aucune mémoire ; le chatbot renvoie tout l'historique, et chaque tour coûte plus.
5. Du JSON se lit par programme, dans un try, avec une nouvelle demande si ça rate ; son contenu se vérifie quand même.

Corrigé

La consigne est-elle tenue ?

1. **Tenue** · Une phrase, et la bonne réponse. Une question simple et une consigne courte : le petit modèle s'en sort.
2. **Oubliée** · Pas un « Arrr », pas un mot de pirate : il répond comme un assistant ordinaire. À 0,5 milliard de paramètres, le rôle ne tient pas.
3. **Oubliée** · La réponse finit sur un conseil, sans question. Il vouvoie aussi, alors que le prompt demandait de tutoyer.
4. **Tenue** · Rien autour des accolades, et `json.loads` le lit du premier coup. Le contenu, lui, est faux : Pikachu n'est pas de type Eau. Une consigne de forme ne garantit pas le fond.
5. **Oubliée** · Le JSON se lit, mais la liste « choix » n'a que deux éléments au lieu de trois. Un programme qui affiche trois choix plantera.
6. **Tenue** · Deux phrases, en français : la consigne de forme est tenue. Il reprend quand même « Nova » comme son propre prénom, une autre histoire.

Mini-quiz

1. **B. Dans le message system** · Le message `system` passe avant tout le reste et l'utilisateur ne le voit pas : c'est la fiche de poste. `user`, c'est toi ; `assistant`, ce que le modèle a déjà répondu.
2. **B. Le programme renvoie toute la conversation à chaque tour** · Le modèle repart de zéro à chaque appel. La classe `Chatbot` ajoute chaque message à sa liste et renvoie la liste entière : c'est elle, la mémoire.
3. **B. L'historique entier est renvoyé à chaque tour, et il grandit** · Chaque tour ajoute une question et une réponse à ce qu'on renvoie. Une longue conversation coûte de plus en plus cher, et un petit modèle finit par perdre le fil.
4. **B. Parce que le modèle ajoute parfois du texte autour, qui ferait échouer `json.loads`** · « Voici le JSON : {...} » n'est pas du JSON valide. On isole la partie entre accolades, on essaie `json.loads` dans un `try`, et on redemande si ça rate.